Fondements informatiques I

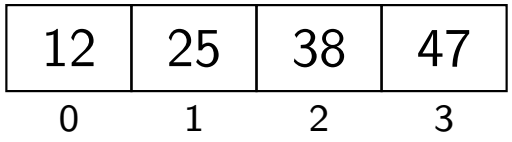
Cours 7: Types et typage

Sorina Ionica sorina.ionica@uvsq.fr

Bilal FAYE bilal.faye@uvsq.fr

Les séquences : définition et types

Une **séquence** est un ensemble fini et ordonné d'éléments. Chaque élément possède une position appelée **indice**, qui commence à 0.



Indices des éléments

En Python, on rencontre notamment quatre types de séquences :

- **Listes** : modifiables (`list`).
- **Tuples** : non modifiables (`tuple`).
- **Chaînes de caractères** : non modifiables (`str`).
- **Intervalles** : séquences d'entiers (`range`).

Exemples

```
l = [12, 25, 38, 47]
t = (12, 25, 38, 47)
s = "Python"
r = range(4)
```

Les séquences : opérations communes

Les mêmes opérations peuvent être utilisées sur les différents types de séquences.

Opération	**Signification**
`len(s)`	Nombre d'éléments de `s`
`s[i]`	Élément d'indice `i`
`s[i:j]`	Éléments de `i` à `j-1`
`s[i:j:k]`	Même principe, avec un pas de `k`
`x in s`	Vérifie si `x` appartient à `s`
`s + t`	Concaténation de deux séquences
`s * n`	Répétition de la séquence `n` fois

Exemple en Python

```
s = [10, 20, 30, 40, 50]

print(s[0])          # 10
print(s[1:3])        # [20, 30]
print(len(s))        # 5
print(30 in s)       # True
```

Les séquences : parcours avec une boucle

Les séquences sont des objets **itérables** : on peut parcourir leurs éléments un par un. La boucle `for` permet de réaliser ce parcours.

Exemple en Python

```
# Une chaine de caracteres
ma_chaine = "Python"
for carac in ma_chaine:
    print(carac)
# Une liste
l = [55, 23, 103]
for nb in l:
    print(nb)
# Un tuple
t = (128, 0, 255)
for nb in t:
    print(nb)
```

La boucle `for` parcourt les éléments sans avoir besoin de manipuler leurs indices.

Chaînes de caractères

Les chaînes de caractères sont des séquences : on peut accéder à leurs caractères par indice et utiliser des tranches.

Accès aux caractères et tranches

```
chaine = 'Abracadabra!'
print(chaine[2])      # r
print(chaine[2:5])    # rac
print(chaine[-1])     # !
```

Une particularité : les chaînes sont immuables

- Une chaîne ne peut pas être modifiée après sa création.
- On ne peut donc pas remplacer directement l'un de ses caractères.

Une modification interdite

```
chaine[2] = 'd'  # TypeError
liste = ["Paris", "Londres", "Rome"]
liste[1] = "Lille"          # Autorise : la liste est modifiable
liste[1][0] = 'l'           # TypeError : la chaine ne l'est pas
```

Des caractères aux nombres

- Un ordinateur manipule des nombres : pour représenter du texte, il faut associer un nombre à chaque caractère.
- L'**ASCII** est un premier système : il associe 128 caractères à des nombres de 0 à 127.
- **Unicode** étend ce principe à un très grand nombre de caractères (accents, alphabets, symboles, etc.).

En Python : caractères et nombres

- `ord(c)` donne le point de code Unicode du caractère `c`.
- `chr(n)` donne le caractère associé au point de code `n`.

Observer la correspondance

```
print(ord('A'))   # 65
print(chr(65))    # 'A'
print(ord('é'))   # 233
print(chr(233))   # 'é'
```

Du texte aux octets

- En Python, une chaîne (`str`) est une séquence de caractères Unicode.
- Pour enregistrer ou transmettre du texte, il faut le convertir en octets : c'est le rôle de l'**encodage**.

Encoder et décoder en UTF-8

```
texte = "é"
octets = texte.encode("utf-8")
print(octets)  # b'\xc3\xa9'
texte_retrouve = octets.decode("utf-8")
print(texte_retrouve)  # é
```

$$\texttt{str} \xrightarrow{\texttt{encode}} \texttt{bytes} \qquad \texttt{bytes} \xrightarrow{\texttt{decode}} \texttt{str}$$

- UTF-8 est un encodage qui permet de représenter les caractères Unicode sous forme d'octets.

Les ensembles

Un **ensemble** (`set`) est une collection d'éléments dans laquelle chaque élément apparaît une seule fois.

Exemple

```
A = {1, 3, 10, 5, 2}
B = {1, 10, 3, 3, 5, 2, 1}

print(A)
print(B)
print(A == B)   # True
```

- Les doublons sont automatiquement supprimés.
- Les éléments ne sont pas ordonnés.
- Un ensemble est modifiable : on peut ajouter ou supprimer des éléments.

Créer des ensembles

On peut créer un ensemble directement avec des accolades ou à partir d'un objet itérable avec la fonction `set()`.

Différentes façons de créer un ensemble

```
# Avec des accolades
a = {1, 2, 3, 4}

# A partir d'une liste
b = set([1, 3, 4, 3, 1])

# A partir d'une chaine
c = set("Python")

# A partir de range()
d = set(range(5))
```

Attention

Pour créer un ensemble vide, on utilise `set()`.
Les accolades vides $\{\}$ créent un dictionnaire.

Parcourir un ensemble

Un ensemble est **itérable** : on peut parcourir ses éléments avec une boucle `for`. En revanche, il n'est pas ordonné : on ne peut pas accéder à un élément à partir de sa position.

Parcourir un ensemble

```
nombres = {10, 20, 30, 40}

for element in nombres:
    print(element)
```

Attention

L'instruction `nombres[0]` provoque une erreur : un ensemble ne permet pas l'accès par indice.

Supprimer les doublons

Les ensembles sont particulièrement utiles pour supprimer les doublons d'une liste.

Problème

Écrire une fonction qui reçoit une liste et renvoie une nouvelle liste contenant les mêmes éléments, sans doublons.

Supprimer les doublons

Les ensembles sont particulièrement utiles pour supprimer les doublons d'une liste.

Problème

Écrire une fonction qui reçoit une liste et renvoie une nouvelle liste contenant les mêmes éléments, sans doublons.

Une solution avec les ensembles

```
def supprime_doublons(liste):
    return list(set(liste))

nombres = [1, 3, 3, 5, 6, 7, 3, 1]

print(supprime_doublons(nombres))
```

Remarque

L'ordre des éléments n'est pas nécessairement conservé.

Opérations sur les ensembles

Les ensembles permettent de réaliser plusieurs opérations mathématiques.
Soient deux ensembles :

$$A = \{0, 1, 2, 3, 4\}, \qquad B = \{2, 3, 4, 5\}$$

- **Union** : éléments appartenant à A ou à B.

$$A \cup B = \{0, 1, 2, 3, 4, 5\}$$

- **Intersection** : éléments communs à A et B.

$$A \cap B = \{2, 3, 4\}$$

- **Différence** : éléments appartenant à A mais pas à B.

$$A \setminus B = \{0, 1\}$$

- **Différence symétrique** : éléments appartenant à un seul des deux ensembles.

$$A \triangle B = \{0, 1, 5\}$$

Opérations sur les ensembles en Python

Python propose des opérateurs pour réaliser les opérations sur les ensembles.

Opérateur	Opération
|	Union
&	Intersection
-	Différence
^	Différence symétrique

Exemple

```
A = {0, 1, 2, 3, 4}
B = {2, 3, 4, 5}

print(A | B)   # Union
print(A & B)   # Intersection
print(A - B)   # Difference
print(A ^ B)   # Difference symetrique
```

Les dictionnaires

Un **dictionnaire** (`dict`) est une structure de données qui associe une **clé** à une **valeur**.

- Chaque élément est un couple `clé : valeur`.
- On accède à une valeur grâce à sa clé, et non par sa position.
- Un dictionnaire est modifiable.

Exemple : notes d'élèves

```
notes = {
    "Gaetan": 14,
    "Laure": 9,
    "Kenza": 17,
    "Gabriel": 14
}

print(notes["Laure"])  # 9
```

Créer un dictionnaire

On utilise des accolades {} pour créer un dictionnaire. Chaque entrée est composée d'une clé et d'une valeur, séparées par deux-points.

Créer un dictionnaire

```
eleve = {
    "nom": "Martin",
    "age": 18,
    "note": 15
}

print(eleve)
```

Dictionnaire vide

On peut également créer un dictionnaire vide :

```
dico = {}
```

Clés et valeurs

Dans un dictionnaire :

- Les **clés sont uniques** : une même clé ne peut désigner plusieurs entrées.
- Les **valeurs peuvent être répétées**.
- Les clés doivent être des objets **hachables**, comme les entiers ou les chaînes de caractères.

Exemple

```
notes = {
    "Gaetan": 14,
    "Laure": 14,
    "Kenza": 17
}
```

Attention

Si une clé est utilisée plusieurs fois lors de la création d'un dictionnaire, seule sa dernière valeur est conservée.

Accéder aux valeurs

Pour accéder à une valeur, on utilise sa clé entre crochets.

Accéder à une valeur

```
notes = {"Gaetan": 14, "Laure": 9, "Kenza": 17}

print(notes["Kenza"])  # 17
```

- Si la clé existe, Python renvoie la valeur associée.
- Si la clé n'existe pas, Python déclenche une `KeyError`.

Vérifier l'existence d'une clé

```
if "Kenza" in notes:
    print(notes["Kenza"])
```

Modifier un dictionnaire

Un dictionnaire est **modifiable** : on peut ajouter, modifier ou supprimer des entrées.

Ajouter et modifier une entrée

```
notes = {"Gaetan": 14, "Laure": 9}

# Modifier une valeur existante
notes["Gaetan"] = 12

# Ajouter une nouvelle entrée
notes["Kenza"] = 17

print(notes)
```

À retenir

- Clé existante : la valeur est remplacée.
- Nouvelle clé : une entrée est ajoutée.

Parcourir un dictionnaire

Une boucle `for` permet de parcourir les éléments d'un dictionnaire.

Parcourir les clés

```
for cle in notes:
    print(cle)
```

Parcourir les valeurs et les couples

```
# Parcourir les valeurs
for valeur in notes.values():
    print(valeur)

# Parcourir les couples cle-valeur
for cle, valeur in notes.items():
    print(cle, ":", valeur)
```

Le typage dynamique

En Python, le typage est **dynamique** : le type d'une variable est déterminé automatiquement à partir de la valeur qui lui est affectée.

Exemple

```
a = 5
print(type(a))   # <class 'int'>

a = "bonjour"
print(type(a))   # <class 'str'>
```

- Une variable peut recevoir des valeurs de types différents.
- Il n'est pas nécessaire de déclarer son type à l'avance.
- Les opérations doivent néanmoins être compatibles avec les types des valeurs manipulées.

Attention

Le typage dynamique ne signifie pas que Python accepte toutes les opérations entre types différents.

Les opérations et les types

Une opération doit être compatible avec les types des valeurs manipulées.

Exemples

```
a = 5
b = 7
print(a + b)          # 12

x = "salut"
y = "hello"
print(x + y)          # saluthello

print(a + y)          # TypeError
print(y / 2)          # TypeError
```

- Deux entiers peuvent être additionnés.
- Deux chaînes peuvent être concaténées.
- Certaines opérations entre types différents ne sont pas autorisées.

Les annotations de type

Python permet d'ajouter des **annotations de type** pour indiquer le type attendu d'une variable.

Exemple

```
age: int = 18
nom: str = "Alice"
taille: float = 1.72

notes: list[int] = [12, 15, 18]
```

- `int` : entier.
- `float` : nombre décimal.
- `str` : chaîne de caractères.
- `list[int]` : liste d'entiers.

À retenir

Les annotations sont facultatives : elles aident à comprendre le programme, mais ne modifient pas le fonctionnement normal de Python.

Annoter les paramètres d'une fonction

On peut préciser le type attendu pour chaque paramètre d'une fonction.

Exemple

```
def addition(a: int, b: int):
    return a + b

print(addition(5, 7))
```

- `a: int` indique que le paramètre `a` est attendu comme un entier.
- `b: int` indique la même chose pour `b`.

Attention

Python ne vérifie pas automatiquement ces annotations à l'appel de la fonction.
Elles servent notamment à guider le programmeur et les outils de vérification de types.

Annoter le type de retour

On peut également indiquer le type de valeur renvoyé par une fonction.
L'annotation `-> int` indique que la fonction est censée renvoyer un entier.

Exemple

```
def somme(a: int, b: int) -> int:
    return a + b

resultat = somme(5, 7)
print(resultat)  # 12
```

Vérification des types

Les annotations ne sont pas vérifiées automatiquement par Python.
Un outil comme `mypy` permet d'analyser le programme et de détecter certaines incohérences de types.